



CartPace

Build Specification and Phased Development Plan

A capacity-aware online controller for tuberculosis screening under a hard cartridge budget, with per-stratum recalibration learned from censored, delayed confirmatory feedback. This document specifies the complete build across fourteen gated phases, the patent strategy for an India provisional filing, and the evidence required to support it.

Field	Value
Project	CartPace -- capacity-aware consumable allocation controller
Document	Build specification, Phase 0 through Phase 13
Primary goal	India provisional patent application, self-drafted
Secondary goal	Demo-quality full-stack web application (course deliverable)
Classification	Complex / full-stack / modular development
Timeline	Approximately four weeks, gated per phase
Dataset policy	Public datasets only; synthetic scores for the core contribution
Status	Rebuild. Supersedes the lost first iteration.
Revision	Rev 1.0 -- 10 September 2026

Contains unfiled patent subject matter. Do not publish, preprint, present or push public before the filing date exists.

Contents

1. Executive Summary	3
2. Phase 0 -- Project Input and Requirements	6
3. Phase 1 -- Initialize and Inspect	10
4. Phase 2 -- Project Analysis	12
5. Phase 3 -- ML and AI Analysis	15
6. Phase 4 -- Software Architecture	17
7. Phase 5 -- Implementation Strategy	21
8. Phase 6 -- Risk Spike and the Go/No-Go Gate	25
9. Phase 7 -- Development and Testing	27
10. Phase 8 -- Integration and End-to-End	29
11. Phase 9 -- Debugging and Quality	30
12. Phase 10 -- Documentation	31
13. Phase 11 -- Final Audit	32
14. Phase 12 -- Clean Build and Deployment Check	34
15. Phase 13 -- Git, Delivery and Verification	35
16. Patent Strategy	36
17. Schedule	39
18. Risk Register	40
19. Appendix A -- Findings of the earlier risk spike	41
20. Appendix B -- Consolidated Definition of Done	43

1. Executive Summary

What this document is, and the one decision it exists to protect

This is the build specification for **CartPace**, a capacity-aware controller for tuberculosis screening at a primary health centre. It is organised as fourteen phases (Phase 0 through Phase 13) following the project's standing development workflow. Each phase ends in an explicit, testable exit gate. No phase begins before the previous gate passes.

The document serves three audiences at once: the engineer building it, the examiner reading the patent specification that will be drafted from it, and the evaluator assessing it as a course deliverable.

The problem in one paragraph

A primary health centre screens roughly 200 people per day for tuberculosis and holds roughly 40 GeneXpert cartridges to confirm diagnoses. A screening model produces a risk score per patient; confirmation consumes one cartridge. The referral threshold is fixed by protocol and never adapts to today's actual capacity. Set it too low and cartridges are exhausted by mid-morning, so afternoon arrivals are turned away and lost to follow-up, which in TB care means untreated infectious disease and continued community transmission. Set it too high and cartridges expire unused while catchable cases walk out of the clinic.

Three constraints that make it hard

- **The budget is hard and depletes irreversibly.** A cartridge spent at 09:00 is unavailable at 16:00.
- **Decisions are online and irreversible.** The patient is physically present. You decide now. You cannot recall them after seeing the rest of the day's cohort.
- **Feedback is censored and delayed.** Confirmatory results return days later, and only for patients who were referred. The system never directly observes what it missed.

The third constraint is the deep one. An allocation system that only ever sees labels for patients it referred cannot, by construction, measure its own false-negative rate.

Why this is a rebuild, not a continuation

An earlier iteration of this project existed and has been deliberately restarted. Two reasons, both material to how this document is written.

The previous repository was lost. The working tree and the Git history were both destroyed by a cloud-sync event. No remote existed. Consequently, Phase 1 of this plan establishes an offsite remote before a single source file is written, and this requirement is non-negotiable.

More importantly, the earlier design produced a structural null result that could not be fixed by tuning. A risk spike measured 200 evaluation days across 5 seeds and found that a pure budget-pacing ablation outperformed the full three-mechanism controller in every regime tested. The conformal-prediction layer, which was supposed to be the contribution, contributed nothing at all.

The result that reshaped the design. Within a single day, with a hard count budget B and a single calibrated monotone score, the optimal referral policy is exactly top- B by score. The ranking is fixed by the score, so any threshold policy is only ever an online approximation of top- B . The entire within-day design space therefore reduces to forecasting today's patient volume -- which is precisely what budget pacing already does. Pacing winning was not a tuning failure. It was the theorem.

Compounding this, the original combination rule was sign-inverted. The conformal threshold updated as $\text{tau_conf} \leftarrow \text{tau_conf} + \gamma * (\alpha - \text{fnr_estimate})$, which **raises** the threshold when the controller is performing well, and the two thresholds were combined as $\text{tau} = \max(\text{tau_conf}, \text{tau_budget})$, which only lets the conformal threshold bind when it is the more conservative of the two. The conformal layer was structurally locked to the useless half of its own range: it could take control only when it wanted to refer fewer people, and only when it believed it was already succeeding.

What the redesign changes

The budget fixes **how many** referrals happen. The only freedom that remains is **which** patients receive them. With a single monotone score there is no such freedom -- unless the score's **ranking** is wrong for some subpopulation. Then there is, and censored feedback can find it.

The redesigned controller maintains a small set of per-stratum calibration offsets, learned online from propensity-weighted delayed feedback, and paces the budget against the corrected score. It changes the **composition** of the referred set without changing its **size**. Four specific corrections, each developed in Phase 2 and Phase 5 of this document:

Correction	Old behaviour	New behaviour
C1 Recalibration	$\max(\text{tau_conf}, \text{tau_budget})$, sign-inverted, no channel to help	K per-stratum logit offsets learned from censored feedback; pacing runs on the corrected score
C2 Drift model	Prevalence drift only, which preserves ranking and is invisible to a quantile method	Ranking degradation on a designated stratum -- the regime where recalibration must prove itself
C3 Primary metric	Cases per cartridge, a gameable ratio that rewards referring almost nobody	Cases caught at matched cartridge spend, reported per scenario
C4 Exploration framing	Exploration "buys a certificate", treated as a cost centre	Exploration is what prevents the control loop from silently collapsing to referring no one

Fixed project decisions

Decision	Value
Primary goal	Patent. An India provisional application, self-drafted.
Secondary goal	A demo-quality full-stack web application, required by the course rubric.
Publication	Secondary, and strictly after the filing date exists.
Timeline	Approximately one month, no hard external deadline.
Clinical collaborator	None. Synthetic patient streams only.
Screening model	Deferred. Synthetic scores first; real chest X-ray data only as a late validation experiment if time permits.
Prior art clearance	Completed before any implementation code is written.

How to read the rest of this document

Chapters 2 through 15 are Phase 0 through Phase 13. Chapter 16 covers patent strategy in full. Chapter 17 is the four-week schedule. Chapter 18 is the risk register. Appendix A preserves the findings of the earlier risk spike, which are the only surviving record of that work and are evidence of inventive step. Appendix B is the consolidated Definition of Done as a single checklist.

2. Phase 0 -- Project Input and Requirements

Requirements capture and the `project.md` contract

0.1 Problem statement

As stated in Chapter 1. Recorded verbatim in `project.md` so that the requirements audit in Phase 11 has a fixed reference to compare against.

0.2 Project objective

Build a **model-agnostic online controller** that wraps any existing frozen screening model -- a chest X-ray CNN, a cough classifier, or a symptom-based risk score -- and for each arriving patient decides refer-or-not so as to:

- maximise tuberculosis cases confirmed for a given number of cartridges spent;
- never exceed the daily cartridge budget, as a hard invariant;
- maintain a continuously updated, distribution-free certificate on the false-negative rate, valid under covariate drift and under censored, delayed feedback;
- detect and correct online degradation in the frozen model's ranking for identifiable subpopulations;
- run in constant memory with bounded per-decision latency, with no retraining and no network connectivity at decision time.

Delivered as a working full-stack web application with a live operations dashboard.

0.3 Project description

The controller sits between a screening score and a referral decision. It holds three pieces of state, all of bounded size, and it is the only component that needs to be novel.

Budget pacing. A bounded quantile sketch over recent corrected scores estimates the score distribution. The pacing threshold is set to the quantile that spends the remaining budget evenly across the arrivals still expected today. Today's volume is re-estimated continuously from today's own arrival rate against the clock, which is observable in any clinic, so the estimate self-corrects within the day rather than relying on a historical average.

Per-stratum recalibration. Patients are partitioned into a small fixed number of coarse strata on covariates available at the point of care. One scalar offset per stratum corrects the frozen model's score in logit space. The offsets are learned from delayed confirmatory labels using propensity weighting, so they remain valid despite the fact that labels only ever arrive for referred patients.

Self-funded exploration. Deterministic thresholding assigns every sub-threshold patient a referral probability of exactly zero -- not small, zero. Positivity fails, inverse-propensity weighting is undefined, and no reweighting can recover the missing mass. The controller therefore refers stochastically: patients below the threshold are referred with a small, deliberately chosen, logged probability. Those exploratory referrals are paid for out of the same fixed cartridge budget the controller is optimising.

0.4 Required features

ID	Requirement
FR1	Ingest a stream of patients with screening scores and stratum covariates, one at a time, in arrival order.
FR2	Emit an immediate, irreversible refer or no-refer decision per patient, with the logged propensity and a human-readable reason.

ID	Requirement
FR3	Never exceed the configured daily cartridge budget. Hard invariant, enforced in the controller and asserted in tests.
FR4	Accept confirmatory labels arriving on a configurable delay, for referred patients only, each carrying the propensity that produced its referral.
FR5	Maintain and expose a live false-negative-rate certificate with a variance estimate, updated as delayed labels land.
FR6	Maintain and expose the per-stratum calibration offsets and their effective sample sizes.
FR7	Run five baseline policies on the identical seeded stream for comparison.
FR8	Persist every run, decision, and label event; support full deterministic replay.
FR9	Stream decisions to the browser live over WebSocket.
FR10	JWT authentication with two roles; supervisor-only routes for budget and scenario control.
FR11	Configurable scenarios: prevalence, arrival profile, drift mode, budget, feedback delay, shortage probability.
FR12	Report score-model quality: AUC, reliability curve, dataset provenance and licence.
FR13	Benchmark and report controller memory footprint and per-decision latency.

0.5 Non-functional requirements

ID	Requirement
NFR1	Controller memory is $O(1)$ in the number of patients seen -- flat from 1,000 to 100,000 decisions.
NFR2	Per-decision latency is bounded and measured in microseconds.
NFR3	Runs fully offline. No network dependency at decision time.
NFR4	Reproducible: seeded runs produce byte-identical results.
NFR5	Clean-environment install from the README succeeds without a GPU and without model weights.
NFR6	No secrets in the repository.
NFR7	The controller package imports nothing but numpy, and never imports from the backend.

0.6 Inputs and outputs

Inputs. Patient screening score, or a chest X-ray image scored locally; stratum covariates (age band, sex, HIV status, prior-TB history, symptom count); daily cartridge budget; scenario configuration; delayed confirmatory results.

Outputs. Per-patient refer or no-refer decision with propensity and reason; live false-negative-rate certificate with interval; per-stratum offsets; cases caught at matched spend; budget burn-down against the clock; baseline comparison table; memory and latency benchmark.

0.7 User roles

Role	Sees	Can do
health_worker	Live decision feed, per-patient decision and reason, patient history	Run the clinic day, record outcomes
supervisor	Everything above, plus baseline comparison and scenario controls	Set cartridge budget, configure scenarios, run comparisons, export results

0.8 Constraints

- No local GPU. Any training is Colab-only.
- The budget is a hard physical constraint and always wins over the false-negative-rate target.
- Decisions are online and irreversible. No lookahead is permitted to any policy except the declared oracle baseline.
- Labels are censored to referred patients and arrive delayed.
- No clinical collaborator and no real clinic data. Arrival streams and capacity are simulated.

0.9 Assumptions

- Simulated arrival streams and capacity are acceptable in place of live clinic data for the purposes of both the patent specification and the course deliverable.
- Stratum covariates are available at the point of care without connectivity. This is true of age, sex, HIV status and symptom count in Indian PHC practice.
- Feedback delay is configurable; a fixed three-day delay is the default, variable delay is supported.
- Single clinic, single device. No multi-site concurrency.

0.10 Out of scope for version 1

Naming what is deliberately not being built is what stops the project becoming maximal. Each exclusion carries the trigger that would pull it into version 2.

Excluded	Trigger to add
Multi-clinic or multi-tenant operation	A second real deployment site exists
PostgreSQL	Concurrent writers appear
Cough audio and EHR modalities	The chest X-ray arm is fully validated and time remains
Real deployment to clinic hardware	Patent filed and a partner site confirmed
Mobile application	A field user asks for it
Patient queueing and overflow to the next day	The single-day model is proven first
Model explainability and saliency maps	A clinical reviewer requests it
Monthly budget pooling across days	The daily controller passes its gate (see Phase 7)

0.11 Riskiest assumptions

These drive the Phase 6 risk spike. If any is false, the project as specified fails.

- **R1. Per-stratum recalibration may have no measurable effect either.** The earlier design's conformal layer had no channel through which to improve outcomes. If recalibration also fails to beat a pure pacing ablation, the contribution does not exist.
- **R2. Exploration cost may exceed its benefit.** Cartridges spent below the threshold are cartridges not spent on likely cases.
- **R3. The propensity-weighted estimates may be too high-variance to steer on.** Small propensities produce large weights and unstable estimates, and the problem worsens when the estimate is split across K strata instead of pooled.
- **R4. Prior art may anticipate the claim.** Contextual Bandits with Knapsacks addresses budget-constrained online decisions via primal-dual thresholding and may read directly onto a budget-coupled-threshold claim.

0.12 Exit gate for Phase 0

project.md exists and contains every subsection above, including the out-of-scope list and the riskiest assumptions. A reader who has never seen this project can state what is being built, what is not, and what would prove it wrong. **No implementation begins until this is true.**

3. Phase 1 -- Initialize and Inspect

Environment, repository safety, skills verification

1.1 Repository and offsite backup -- do this first

The previous repository was destroyed by a cloud-sync event with no remote. This step exists because of that, and it precedes every other action in the project.

```
git init
git add .gitignore README.md project.md
git commit -m "chore: scaffold CartPace"
git remote add origin https://github.com/ishitasolanki/CartPace.git # private repository
git push -u origin main
```

Two further decisions:

- **The project stays in the OneDrive-synced Desktop folder.** This is a settled decision, taken with the earlier loss in full view, and it is not to be revisited. The consequence is that the **GitHub remote is the only real copy of this project**. Every working session must therefore end in a push, and no work may be left uncommitted overnight. An uncommitted change in this folder is one sync event away from gone.
- **.gitignore must exclude** `node_modules/`, `.venv/`, `__pycache__`, `*.pt`, `*.pkl`, `.env`, and `data/raw/`. This protects both Git hygiene and, where the folder remains synced, the sync client.

1.2 Environment verification

Verify, do not assume:

Item	Check	Expected
Python	<code>python --version</code>	3.11 or later
Node	<code>node --version</code>	20 or later
GPU	<code>python -c "import torch; print(torch.cuda.is_available())"</code>	False -- confirms the Colab-only training constraint
Package manager	<code>pip --version</code> , <code>npm --version</code>	present
Git	<code>git --version</code> , remote reachable	present, push succeeds

Record the results in `docs/environment.md`. A plan that assumes hardware that is not present fails late and expensively.

1.3 Codebase inspection

The project starts from an empty directory, so inspection is brief but not skipped: confirm the directory is genuinely empty, confirm no stale virtual environment or `node_modules` survives from the previous iteration, and confirm nothing in the parent directory will be accidentally captured by a broad `git add`.

1.4 Skills and tooling verification

Detect, test, confirm, then use. Do not assume a capability exists because its name appears somewhere.

Capability	Verification
numpy	Import and run a seeded random draw
pytest	Run a trivially passing test, then a deliberately failing one
reportlab	Already confirmed present at version 5.0.1, used to produce this document
FastAPI and Uvicorn	Start a hello-world app, hit it with a request
React, Vite, TypeScript	<code>npm create vite</code> , confirm dev server starts
Workflow discipline skills	Confirm the project workflow skill is loaded and current

1.5 Exit gate for Phase 1

Remote exists and a test commit has been pushed and verified on the remote. Python, Node and Git versions recorded. GPU absence confirmed and recorded. Every tool in the stack has been started at least once. **A clean clone of the remote into a fresh directory succeeds.**

4. Phase 2 -- Project Analysis

Acceptance criteria, the metric trap, datasets, classification

2.1 The primary metric, and why the obvious one is wrong

This subsection is the most important in Phase 2, because the earlier iteration encoded a gameable metric directly into its Definition of Done and then tuned against it.

Cases per cartridge is a gameable ratio. Refer only the single highest-scoring patient each day and the ratio approaches 1.0 while the clinic catches almost nobody. The earlier spike demonstrated this concretely: the full-lookahead oracle appeared to **lose** on cases per cartridge, scoring 0.416 against the controller's 0.431, while catching 310 more cases. A metric on which the theoretical optimum loses is not a metric.

Primary metric. Cases confirmed, with cartridges spent always reported alongside. Where two policies spend different amounts, compare cases at matched spend. Never report a ratio without its denominator. This is the number that decides whether the project worked, and no acceptance criterion may be stated in terms of an unaccompanied ratio.

Secondary metrics: realised false-negative rate against certified false-negative rate; per-stratum calibration error; exploration spend as a fraction of budget; budget exhaustion time; memory footprint; per-decision latency.

2.2 Acceptance criteria

Criteria are stated per scenario, because adaptivity is correctly worthless when there is nothing to adapt to. A controller that matches a tuned fixed threshold in a stationary world and beats it in a drifting one is behaving exactly as intended.

ID	Criterion	Measured by
AC1	Budget invariant never violated across at least 100 seeded days, all scenarios	tests/test_invariants.py
AC2	Every logged decision carries a propensity strictly inside (0, 1]	tests/test_invariants.py
AC3	Under ranking_drift, CartPace beats the pacing-only ablation on cases at matched spend, across at least 5 seeds	spike_p2.py, then bench/compare.py
AC4	Under ca_lm, CartPace does not lose to the pacing-only ablation by more than 1 percent	same
AC5	CartPace beats the tuned fixed threshold and the greedy policy in every scenario	same
AC6	Certified false-negative rate brackets the realised false-negative rate in simulation	tests/test_certificate.py
AC7	Memory and latency flat from 1,000 to 100,000 decisions	bench/footprint.py
AC8	Zero-exploration collapse reproduces, and the effective-sample-size guard prevents it	tests/test_stability.py
AC9	A delayed label is always paired with the propensity that produced it	tests/test_label_pairing.py

ID	Criterion	Measured by
AC10	Unauthenticated request returns 401; wrong-role request returns 403; expired token rejected	tests/test_api_auth.py
AC11	Full browser end-to-end: log in, run a day, watch offsets move as labels land	manual plus Playwright
AC12	Clean-environment install from README succeeds with no GPU and no weights	docs/clean-install.md
AC13	No secrets committed	git log -p scan and .env.example only

2.3 Dataset requirement

Dataset required: NO for the core contribution. YES, optionally, for late validation.

The controller is model-agnostic and operates on scores. Synthetic scores drawn from a two-component generative model reproduce a realistic operating regime -- separating negative and positive score distributions to give an AUC of approximately 0.87, which matches published performance for TB chest X-ray AI. The contribution is the controller, not the scorer, so the scorer may be synthetic throughout.

If time remains after the gate in Phase 7, real data enters as a validation experiment only:

Dataset	Source	Licence position	Use
TBX11K	Kaggle, usmanshams/tbx-11	Public, Kaggle API token required	Primary chest X-ray training set
Shenzhen and Montgomery	NIH / Open-i	Open access	Additional data and evaluation

Public datasets only. Licence, version and download date recorded in docs/dataset.md before any use. Two datasets are explicitly rejected: the CODA TB cough challenge requires Synapse certification and a data-use statement, and MIMIC-IV requires CITI training with a one-to-two-week lead time. Neither strengthens the claim, and both are out of scope.

Splits are patient-level, never image-level. Shenzhen and Montgomery contain multiple views per patient. Splitting on images leaks patients across train and test and inflates AUC. Enforced by an automated test that asserts the train and test patient-ID sets are disjoint.

2.4 Project classification

Dimension	Assessment
Type	Full-stack, ML-adjacent, algorithm-heavy
Features	13 functional, 7 non-functional
Modules	7 top-level packages
Source files	Approximately 45
API endpoints	Approximately 12
Database	SQLite, 7 tables
ML components	Optional, deferred; synthetic scorer is the default

Dimension	Assessment
External integrations	None at decision time, by requirement
Authentication	Yes, JWT, 2 roles
Complexity	Complex

Complex plus backend-heavy plus multiple modules means **modular development**, which Phase 5 specifies. It also means the full Phase 11 audit applies rather than a reduced subset.

2.5 Exit gate for Phase 2

Primary metric written down and deliberately attacked -- a degenerate policy has been shown to score badly on it. All thirteen acceptance criteria recorded in `project.md` with the file that will measure each. Dataset decision recorded with justification. Complexity classified, and the modular path selected on that basis.

5. Phase 3 -- ML and AI Analysis

What is learned online, what is not learned at all

3.1 Is training required?

Offline training: NO, for version 1. The screening model is frozen and may be synthetic. It is explicitly not the contribution, and no architecture search is to be performed on it.

Online learning: YES, and it is the entire contribution. The controller learns two things during operation, from censored and delayed feedback, without retraining and without gradients:

- K per-stratum calibration offsets, which correct the frozen model's ranking;
- one conformal threshold scalar, which tracks the false-negative-rate target.

This distinction matters for the patent specification. The invention is not a trained model. It is a control mechanism that operates on top of any trained model, which is what makes the claim model-agnostic and what makes it survive the scorer being replaced.

3.2 The online learning problem, stated precisely

At decision time for patient i the controller observes a score s_i in $(0,1)$ from the frozen model and a stratum index k_i in $\{1..K\}$. It maintains offsets $d_1..d_K$ and forms a corrected score in logit space:

$$s_{\text{tilde}_i} = \text{sigmoid}(\text{logit}(s_i) + d_{\{k_i\}})$$

Logit space is used rather than additive correction on the probability so that the corrected value remains a valid probability in $(0,1)$ without clipping, and so the correction is a proper monotone recalibration map rather than an arbitrary shift.

The controller refers with probability p_i , which is 1 above the threshold, a small banded value just below it, and a global floor p_{floor} elsewhere. The floor is what guarantees positivity: no patient ever has a referral probability of exactly zero, so inverse-propensity weighting is always defined.

After a delay, for referred patients only, a label y_i arrives **carrying** p_i , the propensity that produced that referral. Per stratum, the controller forms a Horvitz-Thompson estimate of the realised positive rate and compares it against the model's own mean predicted probability for that stratum:

```
w_i      = min( 1 / max(p_i, p_floor), 1 / p_floor )      # clipped, bounds variance
phat_k    = sum_{i in k} w_i * y_i / sum_{i in k} w_i
sbar_k    = mean predicted probability for stratum k
d_k       <- clip( d_k + gamma * (logit(phat_k) - logit(sbar_k)), -L, +L )
```

The update steps only when the stratum's effective sample size clears a floor:

```
ESS_k = ( sum_{i in k} w_i )^2 / sum_{i in k} w_i^2
step only if ESS_k >= ESS_min
```

3.3 Why this is the right algorithm, and what was rejected

The selection criterion is fitness for an offline, constant-memory, label-starved device -- not sophistication.

Candidate	Verdict
Per-stratum logit offsets with propensity-weighted updates	Selected. K scalars. No gradients, no retraining, no matrix algebra. Interpretable: each offset is a readable statement that the model under- or over-scores that subpopulation, which makes it directly displayable on the dashboard and directly describable in a patent claim. Degenerates gracefully to pure pacing when no drift is present.
Online retraining of the scorer	Rejected. Requires gradients, weights, and labels at volume. Violates the constant-memory and no-retraining constraints, and cannot run on a PHC device.
Full contextual bandit with linear payoff	Rejected. Maintains a d-by-d matrix, so memory is not O(1) in the feature dimension, and it re-learns the ranking from scratch rather than correcting a frozen model that is already mostly right. Also the closest prior art, which is a second reason to stay away from its formulation.
Isotonic or Platt recalibration, refit periodically	Rejected. Requires retaining a calibration set, which breaks constant memory, and refitting on censored data without propensity correction is biased.
Importance-weighted logistic regression over covariates	Rejected. More parameters than strata, more variance under label starvation, and far less interpretable on a dashboard.

3.4 Data-quality hazards specific to this setting

Conventional ML hygiene checks map onto this problem in unusual ways, and each has a concrete mitigation.

- **Censoring, not missingness.** Labels are absent by mechanism, not at random. Mitigation: propensity weighting with a guaranteed positivity floor. This is the central design constraint, not an afterthought.
- **Class imbalance.** Prevalence is 3 to 25 percent depending on drift phase. Mitigation: metrics are case-count based, never accuracy based.
- **Variance from small propensities.** Weights up to $1/p_{\text{floor}}$. Mitigation: weight clipping, per-stratum effective-sample-size guard, variance reported alongside every certificate.
- **Stratum starvation.** A rare stratum may accumulate too few labels to steer on. Mitigation: the effective-sample-size guard simply does not step that stratum, leaving its offset at its last value. Strata are kept deliberately coarse, K between 6 and 8, to make starvation rare.
- **Feedback loop bias.** The controller's own decisions determine which labels it sees. Mitigation: this is exactly what the exploration floor exists to break, and the zero-exploration collapse in Appendix A is the empirical demonstration of what happens without it.
- **Leakage.** No offline training in version 1, so no train-test leakage is possible. If the chest X-ray arm is built, patient-level splits are enforced by test.

3.5 Exit gate for Phase 3

The online update rule is written down in full, including clipping bounds, the effective-sample-size floor, and the positivity floor. Algorithm selection is documented with the rejected alternatives and the reason for each. Every data-quality hazard has a named mitigation and the file that implements it.

6. Phase 4 -- Software Architecture

Stack, layering, data flow, security

4.1 Technology stack

Layer	Choice	Reason
Controller and simulator	Pure Python, numpy only	Framework-free so it is testable in isolation, portable to a device, and clean to describe in a patent claim
Backend	FastAPI with Uvicorn	Native async for WebSocket streaming; Pydantic validation on every request body
Database	SQLite with SQLAlchemy	Single clinic, single writer. PostgreSQL is out of scope until concurrent writers exist.
Realtime	WebSocket	Live decision feed, authenticated on connect
Authentication	JWT via python-jose, bcrypt via passlib	Two roles, one shared route guard
Frontend	React, Vite, TypeScript	Type safety across the API boundary
Styling	Tailwind CSS	Consistent spacing and colour scale without bespoke CSS
Charts	Recharts	Composable, adequate for the five required charts
ML, if built	PyTorch, torchvision, scikit-learn	Colab GPU training only
Tests	pytest, Vitest, Playwright	Python, component, and browser end-to-end respectively

4.2 Module layout and dependency direction

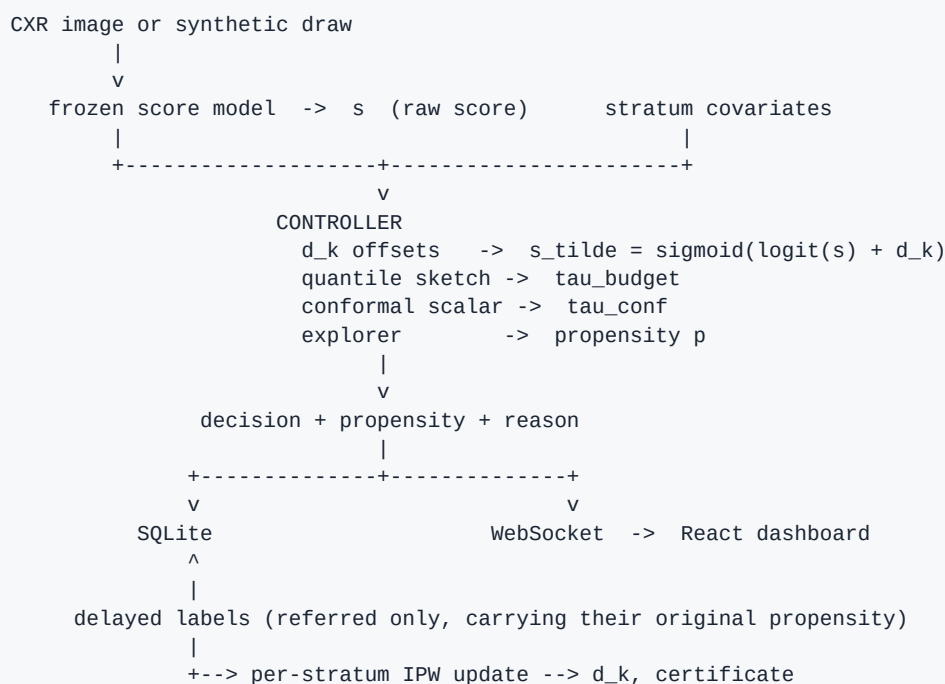
controller/ recalibrate.py conformal.py budget.py explore.py policy.py baselines.py metrics.py	the contribution -- pure, numpy only, no I/O, no framework K per-stratum logit offsets, IPW update, ESS guard, clipping one scalar ACI threshold tracking the FNR target clock-driven pacing, bounded quantile sketch banded propensity, global positivity floor combines the above; owns the hard budget invariant Fixed, Greedy, RandomUnderBudget, ClockPacer, TopBOracle cases at matched spend, realised vs certified FNR, explore cost
sim/ arrivals.py drift.py labels.py scenarios.py	synthetic world -- numpy only morning-weighted inhomogeneous clinic day, seeded prevalence drift AND ranking degradation delayed delivery queue, referred-only, propensity carried calm, prevalence_drift, ranking_drift, shortage, volatile
ml/ backend/ frontend/ bench/ tests/ docs/	optional scorer -- torch; falls back to synthetic FastAPI -- depends on controller, sim, ml React -- depends on backend over HTTP and WS only memory and latency measurement (patent evidence) pytest project.md, modular-plan.md, prior-art.md, dataset.md

Dependency direction is strictly one-way:

```
controller/ -> numpy only
sim/        -> numpy only
ml/         -> torch
backend/    -> controller, sim, ml
frontend/   -> backend (HTTP + WebSocket)
```

controller/ must never import from backend/. This is enforced by a test that walks the import graph. Keeping the controller framework-free is what makes it testable in the Phase 6 spike, portable to clinic hardware later, and clean to recite in a patent claim as a self-contained mechanism.

4.3 Data flow



4.4 Database schema

Table	Key columns	Purpose
users	id, username, password_hash, role	Auth, two roles
runs	id, user_id, scenario, seed, budget, created_at, status	One simulated clinic period
patients	id, run_id, day, arrival_time, stratum, true_label, raw_score	The seeded stream, for replay
decisions	id, patient_id, refer, propensity, tau_conf, tau_budget, s_tilde, reason	Immutable decision log
label_events	id, decision_id, label, propensity_at_decision, arrived_on_day	Delayed feedback; propensity duplicated deliberately
strata_state	id, run_id, day, stratum, offset, ess	Offset trajectory, for the dashboard chart
day_stats	id, run_id, day, caught, spent, budget, explore_spend, fnr_hat, fnr_var	Per-day rollup

`label_events.propensity_at_decision` duplicates a value that also lives in `decisions`. This is deliberate denormalisation: it makes the single most dangerous integration bug in the project impossible to write silently. See Phase 7.

4.5 API surface

Method and path	Role	Purpose
POST <code>/api/auth/login</code>	public	Issue JWT
GET <code>/api/auth/me</code>	any	Current user and role
POST <code>/api/runs</code>	supervisor	Create a run from a scenario and seed
GET <code>/api/runs</code>	any	List runs
GET <code>/api/runs/{id}</code>	any	Run detail and status
POST <code>/api/runs/{id}/start</code>	any	Begin or resume streaming a run
GET <code>/api/runs/{id}/decisions</code>	any	Paged decision log
GET <code>/api/runs/{id}/certificate</code>	any	Current FNR estimate and variance
GET <code>/api/runs/{id}/strata</code>	any	Per-stratum offsets and ESS trajectory
GET <code>/api/runs/{id}/compare</code>	any	Baseline comparison table
GET <code>/api/scenarios</code>	any	Available scenarios and parameters
PUT <code>/api/config/budget</code>	supervisor	Set cartridge budget
WS <code>/ws/runs/{id}</code>	any, authenticated on connect	Live decision stream

4.6 Security design

Designed in now, not retrofitted. Retrofitting guards across twelve existing routes costs more than writing one guard early.

Concern	Measure
Authentication	JWT, short expiry, bcrypt password hashing, no plaintext anywhere
Authorisation	One shared <code>require_role</code> dependency. No route may define its own check, so no route can be silently left unguarded.
Input validation	Pydantic models on every request body and query parameter. Budget and seed bounded.
Injection	SQLAlchemy parameterised queries only. No string-built SQL.
Secrets	<code>.env</code> only, never committed. <code>.env.example</code> carries key names with no values. JWT secret generated at setup.
WebSocket	Token validated on connect, connection rejected before any data is sent
Cross-origin	CORS restricted to the Vite dev origin in development and to nothing in production
Dependencies	Versions pinned in <code>requirements.txt</code> and <code>package-lock.json</code> ; audit run in Phase 11
Sensitive data	Patients are synthetic. No real personal data enters the system in version 1. Recorded as an explicit scope statement.

Concern	Measure
File upload	Chest X-ray upload, if built, restricted by extension, magic-byte check and size cap

4.7 Exit gate for Phase 4

Stack fixed and recorded in `project.md`. Module layout and one-way dependency rule documented. Schema and API surface fully enumerated. Every security measure assigned to a specific file. Nothing here has been implemented yet, which is the point.

7. Phase 5 -- Implementation Strategy

Modular path, and the redesigned controller in full

5.1 Path selection

Backend-heavy, many features, online learning involved, complex architecture, multiple modules, authentication present. **Modular development.** `modular-plan.md` is the artefact; `tasklist.md` is not used.

5.2 The redesigned controller, specified

This is the technical core of both the software and the patent. Four corrections, each traceable to a specific failure of the earlier design.

C1 -- Per-stratum recalibration replaces the max-combination

Strata. K coarse strata, K between 6 and 8, defined on covariates available at the point of care without connectivity: age band, sex, HIV status, prior-TB history, symptom count. **Fixed at configuration time and never learned.** Learning the partition would add parameters, add variance under label starvation, and weaken the claim by making the mechanism less specific.

State. One scalar offset per stratum, plus one conformal scalar, plus a fixed-size quantile sketch. Total state is K plus 2 scalars plus the sketch -- independent of the number of patients seen, so NFR1 holds exactly.

Correction. $s_tilde = \text{sigmoid}(\text{logit}(s) + d_k)$. Budget pacing and thresholding operate on s_tilde , never on the raw score. This is the single most important line in the design: it is what gives the learned component control over **which** patients are referred.

Update. As specified in section 3.2: Horvitz-Thompson estimate per stratum from propensity-carrying delayed labels, compared against the model's own mean predicted probability for that stratum, stepped with gain γ , clipped to a bounded range, and gated on effective sample size.

Why it works where the old design could not. Suppose the frozen model systematically under-scores a stratum -- a CNN undertrained on HIV-positive or paediatric chest X-rays is the canonical real case. Pacing on the raw score will place that entire stratum below the threshold and never refer any of it, so it never generates labels, so nothing ever corrects it. The exploration floor breaks that: a trickle of that stratum is referred anyway, the propensity-weighted estimate reveals that its true positive rate exceeds the model's claim, the offset rises, and the stratum re-enters the referred set -- displacing a genuinely lower-value referral elsewhere.

Total referrals are unchanged. The budget still binds and is still spent in full. What changes is the composition of the referred set. That is the one degree of freedom a hard count budget leaves open, and it is the one the earlier design could not reach.

Graceful degeneration. With no ranking drift, the propensity-weighted estimate agrees with the model, the offsets converge to zero, and the controller reduces exactly to pure pacing. It therefore cannot lose to its own ablation, which repairs the earlier design's 3.1 percent regression in the calm regime.

C2 -- Ranking-degradation drift in the simulator

The earlier simulator drifted prevalence only. Prevalence drift preserves the score ranking, and budget pacing is a quantile method, so it is automatically invariant to prevalence drift. The conformal layer therefore had nothing to detect. **That was a simulator design flaw presenting itself as a negative result about the algorithm**, and it is the most instructive mistake in the project's history.

The new simulator adds a drift mode in which the frozen model's **ranking** degrades: scores for a designated stratum are depressed by a slowly growing logit offset while that stratum's true prevalence holds or rises. Five scenarios:

Scenario	Content	Expectation
calm	No drift, steady volume, steady budget	CartPace approximately equals the pacing ablation
prevalence_drift	Seasonal prevalence, ranking intact	CartPace approximately equals the pacing ablation; both beat fixed
ranking_drift	Score ranking degrades on one stratum	CartPace beats the pacing ablation. This is the gate.
shortage	Budget cut on a fraction of days	Pacing matters most; no regression
volatile	Ranking drift plus supply shocks plus volume swings	Largest margin

C3 -- Primary metric

Cases at matched spend, as established in section 2.1. Cases per cartridge is removed from every acceptance criterion.

C4 -- Exploration as loop stability

The strongest finding of the earlier spike was recorded as a footnote and deserves to be the headline. With zero exploration the propensity-weighted estimator reports a false-negative rate of exactly zero -- because it structurally **cannot observe a miss**. The controller concludes it is performing perfectly and ratchets its threshold upward until it refers almost nobody: 494 cases against 2,868, an 83 percent collapse.

Exploration is therefore not a nicety that purchases a certificate. It is what prevents a closed-loop controller from silently converging on referring no one, and the mechanism of that collapse is specific, reproducible, and non-obvious. This framing is substantially stronger for Indian Patents Act section 3(k) than a constant-memory argument, because it describes a concrete technical problem and a concrete technical solution rather than an implementation efficiency any examiner would call routine.

The collapse becomes a first-class reproducible experiment in `tests/test_stability.py`.

5.3 Module interfaces

```

# controller/recalibrate.py
class StratumCalibrator:
    def __init__(self, n_strata: int, gamma: float, clip: float, ess_min: float,
                  p_floor: float)
    def correct(self, score: float, stratum: int) -> float      # s_tilde
    def update(self, batch: list[tuple[int, float, float, int]]) -> None
        # (stratum, propensity_at_decision, raw_score, label)
    @property
    def offsets(self) -> np.ndarray                            # length K
    @property
    def ess(self) -> np.ndarray                                # length K
    @property
    def state_bytes(self) -> int                               # for the O(1) claim

# controller/budget.py
class BudgetPacer:
    def __init__(self, sketch_size: int = 512, day_hours: float = 8.0)
    def observe(self, s_tilde: float) -> None
    def tau_for(self, budget_left: int, elapsed_fraction: float) -> float
    def end_day(self, n_actual: int) -> None

# controller/explore.py
class Explorer:
    def __init__(self, explore_frac: float, band: float, p_floor: float)
    def propensity(self, s_tilde: float, tau: float, budget_left: int,
                  expected_remaining: float) -> float          # always in (0, 1]

# controller/policy.py
@dataclass(frozen=True)
class Decision:
    refer: bool
    propensity: float
    s_tilde: float
    tau: float
    reason: str      # above_threshold | explore | below_floor | budget_exhausted

class CartPaceController:
    def start_day(self, budget: int) -> None
    def decide(self, score: float, stratum: int, t: float) -> Decision
    def observe_labels(self, batch) -> None                    # delayed, IPW
    def certificate(self) -> tuple[float, float]               # (fnr_hat, halfwidth)

```

5.4 Implementation order

Strictly dependency-ordered, each step tested before the next begins.

Step	Module	Depends on
1	sim/arrivals.py, sim/drift.py, sim/scenarios.py	numpy
2	sim/labels.py	arrivals
3	controller/budget.py	numpy
4	controller/explore.py	numpy
5	controller/recalibrate.py	numpy
6	controller/conformal.py	numpy
7	controller/policy.py	steps 3 to 6
8	controller/baselines.py, controller/metrics.py	policy interface

Step	Module	Depends on
9	spike_p2.py	all of the above -- the gate
10	bench/footprint.py	controller
11	backend/	controller, sim
12	frontend/	backend

5.5 Exit gate for Phase 5

modular-plan.md written, containing the module map, the one-way dependency rule, every interface signature above, the implementation order, and the integration points named in Phase 7. The four corrections are documented with the specific earlier failure each repairs.

8. Phase 6 -- Risk Spike and the Go/No-Go Gate

The phase that can cancel the project

In the standing workflow, Phase 6 is the simple-project path. This project classified as Complex and Modular in Phase 2, so the simple path does not apply. Phase 6 is instead occupied by the risk spike -- which is the correct place for it, because the spike must run **before** the backend and frontend are built on top of an unproven idea.

6.1 Why the spike comes before the application

The earlier iteration's null result was discovered by a spike that cost days. Had the same discovery arrived after the FastAPI backend, the React dashboard and the persistence layer were already built on top of the controller, the sunk cost would have been three weeks and the temptation to rationalise the result instead of fixing the design would have been considerable.

Building in dependency order feels orderly and guarantees that the fatal problem is found last.

6.2 What the spike is

spike_p2.py. A single disposable file. Synthetic scores, no persistence, no API, no UI, no polish. It instantiates the real controller/ and sim/ packages -- not a reimplementation -- and runs the full policy set over seeded multi-day streams in every scenario.

Policies compared:

Policy	Role
Fixed	Current standard of care; threshold tuned on a held-out warmup period
Greedy	High-sensitivity protocol with no pacing; the morning-exhaustion failure mode
RandomUnderBudget	Floor
ClockPacer	The ablation. Budget pacing alone: no recalibration, no exploration, no conformal layer.
CartPace	The full controller
TopBOracle	Full-day lookahead; upper bound

6.3 The ablation requirement

ClockPacer is not an optional extra baseline. It is the ablation that deletes the novel component and keeps everything else intact, and it is the only policy in the table that can tell you whether the contribution is carrying any weight. The earlier iteration survived as a project only because this ablation happened to be written. It is mandatory in every future comparison, and it appears in the dashboard's comparison table so that it cannot quietly disappear from the story.

6.4 The gate, stated as a number before running anything

Condition	Threshold	If it fails
G1	Under ranking_drift, CartPace beats ClockPacer on cases at matched spend by at least 3 percent, mean over at least 5 seeds	Recalibration has no channel either. Stop. Do not build the application on it.

Condition	Threshold	If it fails
G2	Under ca lm, CartPace does not lose to ClockPacer by more than 1 percent	Graceful degeneration is broken; the offsets are not converging to zero. Fix before proceeding.
G3	In every scenario, CartPace beats both Fixed and Greedy	The premise fails. Stop.
G4	Exploration spend stays at or below 10 percent of cartridges at the default setting	Risk R2 has fired. Re-tune the band and quota before proceeding.
G5	The certified false-negative rate brackets the realised rate	Risk R3 has fired. Revisit clipping and the effective-sample-size floor.

6.5 If the gate fails

Failure is an acceptable outcome of this phase and the plan must say what happens next, in advance, so that the decision is not made under sunk-cost pressure.

- **G1 fails.** Fall back to the monthly budget pool as the primary contribution. Lifting the budget from a daily to a monthly allocation changes the structure of the problem: the budget no longer binds within every day, so a shadow price across days becomes meaningful and the certificate becomes the signal for spending more on high-prevalence days and less on quiet ones. This was the earlier iteration's own recommended next step and it remains the strongest fallback.
- **G1 and the fallback both fail.** The patent target is reconsidered before any further engineering. The course deliverable can still be completed honestly as a negative-result study, which is a publishable contribution in its own right at a machine-learning-for-health venue.
- **G4 or G5 fail.** Parameter problems, not design problems. Re-tune and re-run. These do not stop the project.

6.6 Exit gate for Phase 6

`python spike_p2.py` prints a verdict line, and that verdict is **GO** on G1 through G5, reproducibly, over at least 5 seeds. The numbers are committed to `docs/p2-findings.md` together with anything that surprised you. **Phase 7 does not begin until this is true.**

9. Phase 7 -- Development and Testing

Walking skeleton, then the module loop

7.1 Walking skeleton first

Before the module-by-module loop, stand up the thinnest possible end-to-end path: the React login page calls the real POST /api/auth/login, receives a real JWT, the dashboard opens a real authenticated WebSocket, the backend pushes three hard-coded decisions, and the browser renders them in the feed.

Every seam wired. Every layer trivial. Authentication included from the start, because retrofitting guards across twelve routes costs more than writing one guard now.

This finds integration surprises on day two rather than in week three, and it gives every subsequent module a working place to be plugged into.

7.2 The per-module loop

For each module in the Phase 5 implementation order:

- **Look before writing.** Grep the repository and check installed dependencies before adding any helper, type or utility. Re-implementing something that already lives a few files over is the most common failure mode in agent-assisted development.
- **Implement** the minimum that satisfies the interface. No speculative abstraction, no configuration for a value that never changes, no interface with one implementation.
- **Test** expected input, invalid input, edge cases, error handling, and integration with the modules already built.
- **Prove the test can fail.** Break the logic deliberately -- revert the guard, mutate a constant -- watch the test go red, restore it. A test that has never failed has proven nothing.
- **Verify the integrated output**, not just the unit.
- **Commit.** One commit per working, tested module. One commit at the end of a project is unreviewable and leaves no rollback points.

7.3 The three integration points where bugs will actually live

Named in advance, each with a dedicated test.

1. Label-to-propensity pairing. A delayed label must arrive carrying the propensity that produced its referral, not the propensity the controller would assign today. Getting this wrong silently corrupts every estimate, every offset and every certificate, and produces no error -- only quietly wrong numbers. This is why `label_events.propensity_at_decision` is stored denormalised, and it gets `tests/test_label_pairing.py` to itself.

- **2. Controller instance leakage across runs.** The controller is stateful and per-run. Two concurrent runs sharing an instance would cross-contaminate offsets and budget. Test: start two runs with different seeds, assert their decision logs are exactly what each produces in isolation.
- **3. WebSocket reconnect duplication.** A dropped and re-established connection must not replay feed entries the client already holds. Test: force a disconnect mid-run, reconnect, assert no duplicate decision IDs in the client store.

7.4 Test matrix

Level	Coverage
Invariant	Budget never exceeded over at least 100 seeded days; propensity always in (0,1]; memory flat 1k to 100k; controller imports nothing from backend
Unit	Each controller and sim class in isolation, seeded and deterministic
Stability	Zero-exploration collapse reproduces; effective-sample-size guard prevents it
Statistical	Certified false-negative rate brackets realised rate; offsets converge to zero under cal _m ; offsets track the injected degradation under ranking_drift
API	401 unauthenticated, 403 wrong role, expired token rejected, body validation on every endpoint, correct status codes
Frontend	Rendering, forms, validation, navigation, loading states, error states, empty states, responsive behaviour
End-to-end	Login, create run, stream a day, watch offsets move as labels land, certificate updates

7.5 Frontend specification

Three pages, not five. Scenario and budget controls live as a supervisor-only panel inside the live page rather than occupying a route of their own.

Page	Contents
Login	Credentials, validation, error state
Live	Budget burn-down against the clock; both thresholds drawn on the score distribution; per-stratum offsets animating as delayed labels land ; false-negative-rate certificate with its interval; explore-versus-exploit split; patient feed with per-decision reason; supervisor panel for budget and scenario
Compare	Baseline table including the ablation; per-scenario breakdown; cases at matched spend with spend always shown

The offset animation is the centrepiece. It is the mechanism made visible: the operator watches the system discover that the frozen model under-scores a subpopulation and correct for it in real time. It is simultaneously the demo moment, the clearest explanation of the invention, and the source of the best patent figure.

An explicit design pass on typography, spacing, colour and all three of loading, error and empty states is scheduled before the frontend is called done. The rubric asks for good interface quality and a demonstration, so it receives budgeted time rather than whatever is left over.

7.6 Exit gate for Phase 3

Walking skeleton ran end to end before any module was thickened. Every module in the Phase 5 order is implemented, tested, proven-failable, and committed individually. All three named integration points have a passing dedicated test. The three pages render with real data from a real run.

10. Phase 8 -- Integration and End-to-End

Every seam, then the whole path

8.1 Seam verification

Seam	What to verify
Frontend to backend	Token attached on every request; 401 triggers a redirect to login, not a blank screen
Backend to database	Every decision and label event persisted; replay of a seeded run reproduces the original decision log exactly
Backend to controller	Fresh controller instance per run; no state leakage between concurrent runs
Backend to simulator	Delayed labels delivered on the correct day, to the correct run, carrying the correct propensity
Backend to ML	Absent weights degrade to the synthetic scorer with a visible warning, and never crash
WebSocket to client	Authenticated on connect; reconnect without duplication; clean close on run completion

8.2 The end-to-end path

```

health worker
-> Login page, submits credentials
-> POST /api/auth/login, receives JWT
-> Live page opens authenticated WebSocket
-> supervisor creates a run (scenario = ranking_drift, seed = 7, budget = 40)
-> backend streams the simulated day through the controller
-> each decision persisted and pushed to the browser
-> budget burn-down and threshold chart update live
-> day 4: delayed labels from day 1 land
-> per-stratum IPW update fires
-> offsets visibly move on the chart
-> certificate updates with a new interval
-> Compare page shows CartPace against all five baselines

```

Verify the final integrated output, not individual components. The specific thing to watch for is that the offset movement on screen corresponds to labels that actually landed, with the propensities they were actually referred under -- because that is the chain most likely to be subtly wrong while looking entirely plausible.

8.3 Exit gate for Phase 8

Every seam in 8.1 verified by an automated test where possible and by manual observation where not. The full path in 8.2 completed in a real browser against a real backend, and a deterministic replay of the same run reproduces the identical decision log.

11. Phase 9 -- Debugging and Quality

Root cause, reproducibility, performance

9.1 Debugging discipline

For every failure: identify, reproduce, diagnose, **fix the root cause rather than the visible symptom**, retest, and regression-test.

Before editing a shared function, grep every caller. One guard inside the shared function is a smaller change than a guard in each of its callers, and patching only the path the bug report named leaves every sibling caller broken. This project has a specific version of that hazard: the propensity value flows through the simulator, the controller, the database and the WebSocket payload, so a fix applied at one of those four layers may leave the other three wrong.

9.2 Reproducibility verification

Item	Check
Runtime	Python and Node versions pinned and documented
Python dependencies	<code>requirements.txt</code> fully pinned, no loose bounds
Node dependencies	<code>package-lock.json</code> committed
Environment variables	<code>.env.example</code> complete; application fails with a clear message when a required variable is missing, rather than a stack trace
Seeding	The same seed produces a byte-identical decision log on a different machine
Install	Fresh virtual environment from the README succeeds with no GPU and no weights

9.3 Performance

Optimise only what is actually slow, and only where it matters for this project.

Target	Budget
Per-decision controller latency	Microseconds, and flat as the stream grows. This is patent evidence, not a nicety.
Controller memory	Flat from 1,000 to 100,000 decisions, measured and plotted
API response	Under 100 ms for decision-log pages
WebSocket throughput	200 decisions per simulated day streamed without client-side backlog
Frontend	Initial render under 2 seconds; chart updates do not drop frames at streaming rate

9.4 Exit gate for Phase 9

Every known failure fixed at root cause with a regression test. Clean-environment install verified on a fresh virtual environment. `bench/footprint.py` produces the memory and latency figures that will appear in the patent specification, and they are flat.

12. Phase 10 -- Documentation

Describing what was built, not what was planned

Documentation describes the **actual final implementation**. Where implementation diverged from this plan, the documentation records the implementation and the reason for the divergence. A plan presented as a description of something that was built differently is worse than no documentation.

Document	Contents
project.md	Updated to final state; every Phase 0 subsection current
modular-plan.md	Updated module map, interfaces as actually implemented
README.md	What it is, install, configure, run, test, the headline result with its numbers
docs/p2-findings.md	Spike results, the gate verdict, and anything surprising
docs/prior-art.md	Phase 1 clearance analysis and the proceed-or-narrow decision
docs/architecture.md	Diagrams, data flow, schema, dependency rule
docs/api.md	Every endpoint: method, path, role, request, response, status codes
docs/dataset.md	Only if the chest X-ray arm was built: source, licence, version, download date, split policy
docs/environment.md	Verified versions from Phase 1
docs/clean-install.md	The verified clean-environment procedure
docs/patent-evidence.md	Memory and latency figures, ablation results, stability experiment, all with the commands that reproduce them

docs/patent-evidence.md is the bridge between the repository and the provisional specification. Every factual assertion the application makes about the invention's behaviour should be traceable to a command in that file that reproduces the number. An assertion in a patent specification that cannot be reproduced is a liability.

10.1 Exit gate for Phase 10

Every document above exists and describes the built system. A reader with only the repository can install it, run it, reproduce the headline numbers, and understand why the design is what it is.

13. Phase 11 -- Final Audit

Five audits, all of which apply

The project classified as Complex in Phase 2, so the full audit set applies. Nothing here is skippable.

11.1 Requirements audit

Walk FR1 to FR13, NFR1 to NFR7 and AC1 to AC13 one at a time against the built system. For each: implemented, partially implemented, or not implemented, with evidence. Nothing may be silently skipped.

Also audit the out-of-scope list in the opposite direction: did anything creep in that was explicitly excluded? Scope creep is as much a failure as scope shortfall, and unplanned additions carry no tests.

11.2 Code audit

Bugs, dead code, duplication, unused imports, hard-coded values that should be configuration, leftover debug statements and print calls, misleading names, missing error handling, poor organisation. Specific to this project: confirm the controller still imports nothing but numpy, and confirm no deliberate simplification was left in place without a comment naming its ceiling and its upgrade path.

11.3 Machine-learning audit

Check	Requirement
Dataset	Public only, confirmed; licence, version and download date documented
Splits	Patient-level, provably disjoint, enforced by test
Leakage	None; no offline training in version 1, or disjointness proven if the chest X-ray arm was built
Algorithm justification	Documented with rejected alternatives, as in section 3.3
Metrics	Reported with denominators; no unaccompanied ratios anywhere
Reproducibility	Seeded runs byte-identical
Inference	Tested, including the degradation path when weights are absent
Online learning	Offsets converge under <code>calm</code> , track injected degradation under <code>ranking_drift</code> , and never step on a starved stratum

11.4 Interface audit

Visual consistency across the three pages; navigation; responsive behaviour at realistic widths; form validation with useful messages; all three of loading, error and empty states present on every data-bound component; keyboard accessibility and sufficient colour contrast; and overall usability when a person who has not seen the system is asked to run a day unaided.

11.5 Security audit

Re-verify everything designed in section 4.6, now against the built code: no secrets in the repository or in the Git history; `.env` ignored and `.env.example` value-free; authentication and authorisation enforced through the single shared dependency with no route bypassing it; input validation on every endpoint; no exposed debug endpoints; dependency vulnerability scan clean; file upload restrictions in place if upload was built.

11.6 Exit gate for Phase 11

All five audits completed in writing, with findings either fixed or explicitly accepted with a stated reason. The requirements audit shows no silent omissions and no unplanned additions.

14. Phase 12 -- Clean Build and Deployment Check

Proving it works somewhere that is not your machine

12.1 Clean environment test

On a fresh clone into a new directory, with a new virtual environment and no cached artefacts:

```
git clone <remote> cartpace-clean && cd cartpace-clean
python -m venv .venv && . .venv/Scripts/activate
pip install -r requirements.txt
cp .env.example .env          # then fill in the generated secret
pytest                        # all tests pass
python spike_p2.py            # headline result reproduces
python bench/footprint.py     # memory and latency figures reproduce
uvicorn backend.main:app      # backend starts
cd frontend && npm ci && npm run build && npm run dev
```

12.2 Verification checklist

Item	Expected
Install	Succeeds with no GPU and no model weights present
Tests	All pass on the clean clone
Spike	Headline numbers reproduce within seed noise
Backend	Starts, serves, connects to a freshly created database
Frontend	Builds with no type errors and no console errors
Database	Created from scratch on first run; migrations or schema creation succeed
Scorer	Absent weights fall back to synthetic with a visible warning, and do not crash
Features	Every major feature exercised manually once

This phase is where a project that works only on the author's machine is caught. It is also the phase most often skipped, and skipping it is how a demonstration fails in front of an evaluator. Run it on a genuinely clean clone, not on the working directory with files temporarily moved aside.

12.3 Exit gate for Phase 12

The full sequence in 12.1 completed successfully from a fresh clone, and the result recorded in docs/clean-install.md with the date and the machine it was verified on.

15. Phase 13 -- Git, Delivery and Verification

Finishing, and confirming you actually finished

13.1 Repository cleanup

Remove temporary files, scratch scripts, debug output and any dataset or model artefact that should not be version-controlled. Re-read `.gitignore` against what is actually present. Scan the Git history, not only the working tree, for secrets -- a secret committed and later deleted is still in the history and still compromised.

13.2 Review before committing

`git status` and `git diff` reviewed in full. Confirm the correct files, no accidental additions, no secrets, no unrelated changes, documentation included, tests included.

13.3 Commit and push

Commits were made throughout Phase 7, so this is not the first time Git is touched. What remains is the final documentation and audit work, committed with a clear message, then pushed.

Authorship rule. No AI assistant is ever credited in this repository. No co-author trailer on any commit, no generated-with line in any pull request description, no attribution in tags or release notes. The repository is patent evidence and a graded deliverable, so the Git history is a record of authorship and inventorship, not a changelog. This applies to every commit in the project, including those made during Phase 7.

13.4 Verify after pushing

Pushing is not finishing.

- The remote contains the files you expect, and the commit is present on the correct branch.
- A fresh clone of the remote builds and runs -- this is Phase 12 run against the remote rather than against a local copy.
- The README instructions work as written, followed literally, by someone starting from nothing.
- Nothing was accidentally omitted by `.gitignore`. An overly broad ignore rule that excludes a needed source file produces a repository that builds for you and for nobody else.

13.5 Exit gate for Phase 13

A fresh clone of the remote, on a machine that has never held this project, installs and runs by following the README literally, and reproduces the headline result.

16. Patent Strategy

India provisional, self-drafted: clearance, drafting discipline, claim structure

Patent is the primary goal of this project, which reorders several decisions that would otherwise be made differently. This chapter is the reason.

16.1 Prior art clearance comes first

Three days, before any implementation code. If a reference reads onto the intended claim, the claim structure changes, and code written before that is discovered is wasted effort built around an anticipated idea.

Area	Specific references to read
Budgeted online decisions	Contextual Bandits with Knapsacks -- Badanidiyuru, Langford, Slivkins; Agrawal and Devanur. Online decisions under hard resource budgets via primal-dual thresholding. Assume this anticipates "budget-coupled threshold as online constrained optimisation" as a standalone claim.
Propensity-logged decisions	Counterfactual Risk Minimisation and learning from logged bandit feedback -- Swaminathan and Joachims. Covers exploration floors for inverse-propensity validity, generically.
Conformal methods	Adaptive Conformal Inference -- Gibbs and Candes. Conformal Risk Control -- Angelopoulos and others. arXiv 2504.07611 on conditional conformal risk adaptation.
Deferral and selective prediction	Learning to defer under budget -- Mozannar and Sontag.
Recalibration	Online recalibration under distribution shift; active learning with censored or one-sided feedback.
Resource-constrained screening	npj Digital Medicine 2023 on linear-programming optimisation of screening thresholds under resource constraints -- population-level, offline, full labels.
Patent databases	Espacenet, Google Patents, Lens.org, and InPASS for Indian filings. Classifications G16H 40/20 (healthcare resource management), G16H 50/20, G06N. Search terms: test kit allocation, consumable inventory point of care, adaptive referral threshold, reagent budget, cartridge allocation.

Deliverable: docs/prior-art.md, containing an element-by-element comparison against the intended independent claim and an explicit **proceed, narrow, or pivot** decision.

16.2 Section 3(i) -- the larger risk, and it is not about software

Indian Patents Act section 3(i) excludes from patentability any process for the medicinal, surgical, curative, prophylactic, **diagnostic**, therapeutic or other treatment of human beings. A system that decides which patients receive a diagnostic test sits uncomfortably close to this line, and this risk is larger and less appreciated than the software-patentability question.

Drafting discipline, applied from the first document onward. Draft the invention throughout as a **consumable-inventory and resource-allocation controller for a point-of-care testing device**. Never as a diagnostic method, a screening method, or a triage method -- not in the title, not in the claims, not in the abstract, not in the figure captions, and not in the repository's own documentation, which an examiner may well read. The invention allocates a depleting physical consumable. That it happens to be allocated to diagnostic tests is context, not the claimed subject matter.

16.3 Section 3(k) -- technical effect

Section 3(k) excludes a computer programme per se. The established route through it is a demonstrable technical effect beyond the mere execution of a program.

- **Claim the system, not the algorithm.** A system comprising the point-of-care testing device, its cartridge inventory, and the controller. The technical effect is then measurable control over the consumption of a physical consumable, which is a physical outcome.
- **Lead with the stability effect, not the memory footprint.** Constant memory from a fixed-size sketch and a handful of scalars is routine engineering and an examiner will say so. The non-obvious technical problem is that a closed-loop controller learning from censored feedback **silently converges on referring no one**, because its estimator structurally cannot observe its own misses, and the solution is a deliberately funded positivity floor drawn from the same depleting budget. That is a concrete technical problem with a concrete technical solution.
- **Keep the constant-memory and bounded-latency figures as supporting evidence**, measured in bench/, not as the primary argument.

16.4 Inventive step is the weak link, not novelty

Each component of the system is individually known: adaptive conformal inference, inverse-propensity weighting with clipping, quantile-sketch rate control, exploration floors for positivity. An obviousness argument based on combining known components is the most likely objection, and the defence must be specific.

The defensible narrow point is the **coupling**. Exploration that makes censored-feedback recalibration identifiable is paid for out of the same hard, depleting physical budget the controller is optimising, at a measured price, and without it the controller does not merely lose its certificate -- it collapses. Claim the coupling and the collapse, supported by the measured exploration cost and the reproducible 83 percent collapse. Do not claim the three components, separately or as a list.

The per-stratum recalibration from censored feedback is the second differentiator, and it is the one that most clearly distinguishes the invention from Contextual Bandits with Knapsacks: that line of work allocates under budget constraints, but it does not correct a frozen model's ranking from censored, propensity-logged confirmatory feedback.

16.5 Filing mechanics and sequencing

Item	Detail
Route	India provisional application, self-drafted
Official fee	Approximately Rs. 1,600 for an individual applicant (verify the current schedule at filing)
Term	12 months to file the complete specification
Filing	Online via the Indian Patent Office e-filing portal
Prerequisites	Specification, claims, abstract, drawings, Form 1, Form 2, Form 5 where applicable
Drawings	Generated from the dashboard and from bench/ plots

Sequencing is absolute. File before anything becomes public. No preprint, no arXiv submission, no public repository, no demo video, no conference or workshop submission, and no public presentation until the filing date exists. India has no general grace period for prior publication by the applicant that can be relied upon. Publication before filing is an irreversible mistake, and it is the single most common way a student project loses its patent rights.

16.6 Publication, afterwards

Once filed, the publication route is a workshop or conference paper at a machine-learning-for-health venue. The honest framing is strong: the ablation result, the three implementation failures that the earlier spike caught, and the redesign that gave the learned component a channel to act through. Negative and corrective results with a clear mechanism are welcome at these venues, and this project has an unusually clean one.

A clinical journal is not reachable in this cycle. That requires real deployment data or retrospective capacity logs from a real clinic, and there is no clinical collaborator.

17. Schedule

Four weeks, gated

Each week ends at a gate. A failed gate stops the next week rather than being carried forward as a known problem.

Week	Work	Gate
W0, days 1-3	Prior art clearance. No code. docs/prior-art.md with the proceed-or-narrow decision. Repository, remote and project.md established in parallel, since those are safety and not design.	A clearance decision exists in writing, and the claim structure is fixed.
W1	controller/ and sim/ in full, with tests. Then spike_p2.py.	The Phase 6 gate: G1 through G5 pass. Nothing downstream starts until this is true.
W2	bench/footprint.py for the patent figures. Backend: auth, routes, persistence, WebSocket, run engine. Walking skeleton stood up at the start of the week.	Backend tested; skeleton runs end to end in a browser.
W3	Frontend: three pages, the offset animation, the design pass. Integration and end-to-end testing.	Full browser path completes; interface audit passes.
W4	Monthly budget pool, only if the daily controller passed its gate . Provisional specification drafted. Audits, documentation, clean-environment test, demo rehearsal.	Provisional ready to file. Clean clone reproduces the headline result.

Two things are explicitly deferred to "only if time genuinely remains", and neither is permitted to displace the work above: the chest X-ray validation arm with TBX11K, and any additional scenario beyond the five specified.

The monthly budget pool is sequenced after the daily controller deliberately. It is the strongest extension and the natural fallback if the gate fails, but building it first would mean building the harder version of an idea that has not yet been shown to work in its simpler form.

18. Risk Register

What kills this, and what is done about it

Risk	Severity	Mitigation
Prior art anticipates the claim	High	Phase 1 clearance before any code. Outcome is narrow-or-pivot decided on three days of reading rather than four weeks of implementation.
spike_p2 fails -- recalibration also has no channel	High	Hard gate at the end of W1 with the pass condition written as a number in advance. Fallback to the monthly budget pool is specified in section 6.5 before the gate is run, so the decision is not made under sunk-cost pressure.
Section 3(i) objection on diagnostic subject matter	High	Resource-allocation framing applied from day one across every document, including the repository's own.
Inventive-step objection on combining known components	High	Claim the budget-exploration coupling specifically, with the measured exploration price and the reproducible collapse as evidence of non-obviousness.
Publication before filing destroys novelty	High	No public artefact of any kind until the filing date exists. Stated in section 16.5 and repeated in the workflow skill.
Propensity-weighted estimates too noisy per stratum	Medium	Weight clipping, per-stratum effective-sample-size guard, coarse strata (K between 6 and 8), variance reported with every certificate. Gate G5 tests it.
Exploration cost exceeds benefit	Medium	Exploration rate swept in the spike; cost reported as an explicit metric. Gate G4 bounds it at 10 percent of budget.
Application scope consumes the research month	Medium	Three pages, no replay interface, no sweep interface. controller/ and sim/ are complete and gated before any backend work begins.
Data loss recurs	Medium	Remote established in Phase 1 before the first source file. The project stays on the cloud-synced path by decision, so the remote is the only real copy: every session ends in a push, and nothing is left uncommitted overnight.
Demonstration fails on an unfamiliar machine	Low	Phase 12 clean-environment test on a genuine fresh clone, plus a rehearsal in W4.

19. Appendix A -- Findings of the earlier risk spike

Preserved evidence: what was tested, and why the design changed

The earlier iteration's repository was destroyed, and these results are the only surviving record of that work. They are reproduced here for two reasons: they justify every one of the four corrections in Phase 5, and a documented record of what was tried and rejected is evidence of inventive step when the provisional specification is examined.

Conditions: 200 evaluation days, 5 seeds, budget 40 cartridges per day reduced to 25 on the 15 percent of days with a simulated supply shortage, score model AUC approximately 0.87, baselines tuned on a 60-day held-out warmup period that was then discarded.

A.1 Headline results

Policy	Cases caught	Cartridges	Per cartridge	Recall
Greedy, no pacing	1535	7613	0.202	31.0%
Random under budget	856	6843	0.125	17.3%
Fixed, tuned on warmup	2838	6869	0.414	57.3%
ClockPacer, the ablation	2889	6513	0.444	58.3%
CartPace, full controller	2856	6623	0.431	57.7%
Top-B oracle, upper bound	3166	7613	0.416	63.9%

Three readings of this table drove the redesign. The greedy failure mode is real and large, at 86 percent fewer cases, so the motivating problem survives intact. The ablation beat the full controller, so the novel component was contributing nothing. And the oracle appears to lose on cases per cartridge while catching 310 more cases, which condemns the metric.

A.2 Adaptivity pays in proportion to how much the world moves

Scenario	Fixed	Pacer	CartPace	Oracle	CartPace vs fixed
Calm, no drift, steady volume	3045	2980	2950	3228	-3.1%
Baseline, as spiked	2838	2888	2855	3166	+0.6%
Volatile, strong drift and supply shocks	2518	2618	2583	2917	+2.6%

The sign flips with non-stationarity, which is mechanistically correct: adaptivity is a cost when there is nothing to adapt to. The calm-regime regression is what correction C1's graceful degeneration is designed to remove.

A.3 The price of the certificate

Exploration fraction	Cases caught	Exploration as percent of budget	Versus fixed
0.000	2868	2.6%	+1.0%
0.010	2866	3.9%	+1.0%
0.025	2865	5.3%	+0.9%

Exploration fraction	Cases caught	Exploration as percent of budget	Versus fixed
0.050	2856	8.1%	+0.6%
0.100	2829	13.7%	-0.3%

Certification cost approximately 12 cases out of 2,868, or 0.4 percent, at a 5 percent exploration rate. Cheap -- but it purchased a guarantee, not throughput.

A.4 Three implementation failures the spike caught

Each of these is a concrete, non-obvious engineering problem discovered by measurement, and each is worth recording in the patent specification as part of the enablement disclosure.

- **Exploration consumed 21 percent of the budget.** A flat exploration probability applied across the whole population means that probability multiplied by roughly 160 sub-threshold patients per day, not by a handful. Fixed by a bounded per-day quota concentrated in a band immediately below the threshold, where the informative cases actually lie.
- **Pacing against a historical patient count hoards cartridges.** High-volume days exhaust the budget early, and that loss is capped by the budget itself. Low-volume days end with unused cartridges, and that loss is unbounded. The asymmetry produced systematic under-spending, 5,448 cartridges against an available 6,452. Fixed by pacing against the clock and re-estimating today's volume from today's own arrival rate, which self-corrects within the day.
- **Zero exploration destabilises the control loop.** With no exploration the estimator reports a false-negative rate of zero, because it structurally cannot observe a miss. The controller concludes it is performing perfectly and ratchets its threshold upward until it refers almost nobody: 494 cases, an 83 percent collapse. Fixed with an effective-sample-size guard that refuses to steer on a starved estimate.

The last of these is the strongest single piece of evidence for the redesigned claim. Exploration is not merely how the system is certified. It is what prevents the controller from silently converging on referring no one.

20. Appendix B -- Consolidated Definition of Done

One checklist, every gate

The project is finished when every line below is true. Not before, and the absence of any line is not a matter of judgement.

B.1 Gates by phase

- Phase 0: `project.md` complete, including out-of-scope list and riskiest assumptions.
- Phase 1: remote exists, test commit pushed and verified, fresh clone succeeds, environment versions recorded, GPU absence confirmed.
- Phase 2: primary metric written down and attacked; 13 acceptance criteria recorded with their measuring files; dataset decision justified; complexity classified.
- Phase 3: online update rule fully specified; algorithm choice documented with rejected alternatives; every data hazard has a named mitigation.
- Phase 4: stack, module layout, one-way dependency rule, schema, API surface and security measures all documented.
- Phase 5: `modular-plan.md` complete with every interface signature and the implementation order.
- Phase 6: `spike_p2.py` **returns GO on G1 through G5 over at least 5 seeds**, results committed to `docs/p2-findings.md`.
- Phase 7: walking skeleton ran first; every module implemented, tested, proven-failable and committed individually; all three named integration points have passing tests.
- Phase 8: every seam verified; full browser path completes; deterministic replay reproduces the decision log.
- Phase 9: all failures fixed at root cause with regression tests; clean-environment install verified; memory and latency figures flat.
- Phase 10: every document exists and describes the built system, not the plan.
- Phase 11: five audits completed in writing; no silent omissions, no unplanned additions.
- Phase 12: full clean-clone sequence succeeds and is recorded with date and machine.
- Phase 13: fresh clone on an unfamiliar machine installs, runs and reproduces the headline result by following the README literally.

B.2 Acceptance criteria

- AC1 Budget invariant never violated across at least 100 seeded days, all scenarios.
- AC2 Every logged decision carries a propensity strictly inside (0, 1].
- AC3 Under `ranking_drift`, CartPace beats the pacing ablation on cases at matched spend.
- AC4 Under `calm`, CartPace does not lose to the pacing ablation by more than 1 percent.
- AC5 CartPace beats the tuned fixed threshold and greedy in every scenario.
- AC6 Certified false-negative rate brackets the realised rate in simulation.
- AC7 Memory and latency flat from 1,000 to 100,000 decisions.
- AC8 Zero-exploration collapse reproduces; the effective-sample-size guard prevents it.
- AC9 A delayed label is always paired with the propensity that produced it.
- AC10 401 unauthenticated, 403 wrong role, expired token rejected.

- AC11 Full browser end-to-end: log in, run a day, watch offsets move as labels land.
- AC12 Clean-environment install succeeds with no GPU and no weights.
- AC13 No secrets committed, in the working tree or in the history.

B.3 Patent track

- Prior art clearance complete, with a written proceed-or-narrow decision.
- Resource-allocation framing applied consistently across every document, including the repository's own.
- docs/patent-evidence.md complete, with every assertion traceable to a reproducing command.
- Provisional specification, claims, abstract and drawings drafted.
- **Nothing published, preprinted, pushed public, presented or submitted anywhere before the filing date exists.**